DiaCare: Diabetes Prediction Based on Lifestyle Patterns Using Deep Learning

Melisa Lea Andrea
School of Computer Science
Binus University
Bandung, Indonesia
melisa.andrea@binus.ac.id

***Abstract*— Diabetes is a chronic disease with increasing prevalence worldwide and early risk identification plays an important role in prevention efforts. This study aims to develop an artificial intelligence model to classify diabetes risk based on lifestyle and health survey data. This study evaluates Multi-Layer Perceptron (MLP) and XGBoost models for diabetes risk prediction using the imbalanced BRFSS dataset. To address the difficulty of detecting minority cases, the research applied stratified sampling and class weighting. Results indicate that XGBoost achieved a superior recall of 63.5% for the Diabetes class, while MLP performed marginally better for Prediabetes with 30.4% recall. Due to high false-negative rates, the current models are insufficient for definitive clinical diagnosis. However, the study proposes their implementation in "DiaCare," a web-based application designed as a preliminary self-assessment tool to raise user awareness and encourage proactive medical consultation.**

Keywords—diabetes prediction, lifestyle data, class imbalance, artificial intelligence, multilayer perceptron, xgboost

I. Introduction

Diabetes is a major public health issue that can lead to severe complications if not detected and managed early. Lifestyle factors such as physical activity, body mass index, and general health condition are known to be closely related to the risk of developing diabetes [1]. Therefore, predictive models based on health survey data have gained increasing attention in recent years as a tool for early risk classification.

However, a common challenge in diabetes prediction tasks is the imbalance of class distribution, where cases such as prediabetes appear far less frequently than non diabetes cases. This imbalance often causes machine learning models to be biased toward the majority class, resulting in poor detection of high risk but rare conditions [2]. Traditional linear models may also struggle to capture the complex relationships between lifestyle variables and diabetes risk.

To address these challenges, this study proposes the use of a Multi Layer Perceptron and XGBoost based classification model for diabetes risk prediction. The model is trained using stratified data splitting, feature standardization, and class weight adjustment to improve performance on minority classes. The main contribution of this work is the evaluation of recall focused performance on the prediabetes class, highlighting the trade off between minority class detection and overall model accuracy in highly imbalanced health datasets.

II. Methods

A. Dataset Description

The dataset used in this research is taken from Kaggle titled “Diabetes Health Indicators Dataset”[3]. The dataset is taken from The Behavioral Risk Factor Surveillance System (BRFSS) held annually by the CDC. The dataset used in this study contains information related to lifestyle and general health conditions of respondents. It consists of 22 attributes: specifically, 21 features and 1 target attribute. The following are the list of attributes:

TABLE I. Attributes List

Attribute List		
Attribute	**Data Type**	**Information**
HighBP	Binomial	0 = no high BP 1 = high BP
HighChol	Binomial	0 = no high cholesterol 1 = high cholesterol
CholCheck	Binomial	0 = no cholesterol check in 5 years 1 = yes cholesterol check in 5 years
BMI	Float	Body Mass Index
Smoker	Binomial	Have you smoked at least 100 cigarettes in your entire life? [Note: 5 packs = 100 cigarettes] 0 = no 1 = yes
Stroke	Binomial	(Ever told) you had a stroke. 0 = no 1 = yes
HeartDiseaseor Attack	Binomial	coronary heart disease (CHD) or myocardial infarction (MI) 0 = no 1 = yes
PhysActivity	Binomial	physical activity in past 30 days - not including job 0 = no 1 = yes
Fruits	Binomial	Consume Fruit 1 or more times per day 0 = no 1 = yes
Veggies	Binomial	Consume Vegetables 1 or more times per day 0 = no 1 = yes
HvyAlcohol Consump	Binomial	Heavy drinkers (adult men having more than 14 drinks per week and adult women having more than 7 drinks per week) 0 = no 1 = yes
AnyHealthcare	Binomial	Have any kind of health care coverage, including health insurance, prepaid plans such as HMO, etc. 0 = no 1 = yes

NoDocbcCost	Binomial	Was there a time in the past 12 months when you needed to see a doctor but could not because of cost? 0 = no 1 = yes
GenHlth	Integer	Would you say that in general your health is: scale 1-5 1 = excellent 2 = very good 3 = good 4 = fair 5 = poor
MentHlth	Integer	Now thinking about your mental health, which includes stress, depression, and problems with emotions, for how many days during the past 30 days was your mental health not good? scale 1-30 days
PhysHlth	Integer	Now thinking about your physical health, which includes physical illness and injury, for how many days during the past 30 days was your physical health not good? scale 1-30 days
DiffWalk	Binomial	Do you have serious difficulty walking or climbing stairs? 0 = no 1 = yes
Sex	Binomial	0 = female 1 = male
Age	Integer	13-level age category 1 = 18-24 9 = 60-64 13 = 80 or older
Education	Integer	Education level scale 1-6 1 = Never attended school or only kindergarten 2 = Grades 1 through 8 (Elementary) 3 = Grades 9 through 11 (Some high school) 4 = Grade 12 or GED (High school graduate) 5 = College 1 year to 3 years (Some college or technical school) 6 = College 4 years or more (College graduate)
Income	Integer	Income scale scale 1-8 1 = less than $10,000 5 = less than $35,000 8 = $75,000 or more

In addition, the target feature is "Diabetes_012" has 3 varieties of data, which are No Diabetes/Pregnancy, Prediabetes, and Diabetes.

B. Exploratory Data Analysis (EDA)

The whole research will use Google Collab as the main IDE and programmed in Python Programming Languange. EDA is an important step in the data world so that we know the condition of our data better, ensuring that we can make the best model out of our dataset [4]. First, we need to ensure the data distribution. For features those are binomial, in other words owning only 2 types of output will be inferred by count and percentage.

```
Veggies (Nilai Unik):
         Count Percentage
Veggies
0.0      47839     18.86 %
1.0     205841     81.14 %

HvyAlcoholConsump (Nilai Unik):
                 Count Percentage
HvyAlcoholConsump
0.0                239424     94.38 %
1.0                 14256      5.62 %

AnyHealthcare (Nilai Unik):
                Count Percentage
AnyHealthcare
0.0             12417      4.89 %
1.0            241263     95.11 %

NoDocbcCost (Nilai Unik):
               Count Percentage
NoDocbcCost
0.0           232326     91.58 %
1.0            21354      8.42 %

DiffWalk (Nilai Unik):
          Count Percentage
DiffWalk
0.0      211005     83.18 %
1.0       42675     16.82 %

Sex (Nilai Unik):
     Count Percentage
Sex
0.0  141974     55.97 %
1.0  111706     44.03 %
```

Fig. 1. Binary Features Data Distribution

From this input we can see that several features are not distributed equally. Take the feature representing alcohol consumption; only 5.62% of the subjects are heavy drinkers. Next unto non-binary features will be presented by histogram tables.

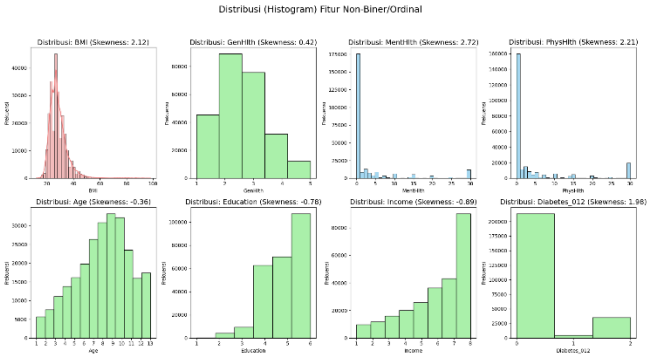

Fig. 2. Non-Binary Features Data Distribution

Here we can see that most features are left or right skewed, and only one seemed to nearly be normally distributed. Through this first step of EDA, the risk of data bias in our model is high since most features don't have normally distributed data.

Second, we will check for missing values.

```
# --- 4. Cek Missing Value ---
print("--- 4. Cek Missing Values ---")
print(df.isnull().sum())
print("-" * 30)
```

```
--- 4. Cek Missing Values ---
Diabetes_012            0
HighBP                  0
HighChol                0
CholCheck               0
BMI                     0
Smoker                  0
Stroke                  0
HeartDiseaseorAttack    0
PhysActivity            0
Fruits                  0
Veggies                 0
HvyAlcoholConsump       0
AnyHealthcare           0
NoDocbcCost             0
GenHlth                 0
MentHlth                0
PhysHlth                0
DiffWalk                0
Sex                     0
Age                     0
Education               0
Income                  0
dtype: int64
------------------------------
```

Fig. 3. Data Missing Value Discovery

Here there is no missing value in any of the features, which is great.

Third, we need to make sure which features effects the target of our variable to see if we can remove any feature to make the model more accurate.

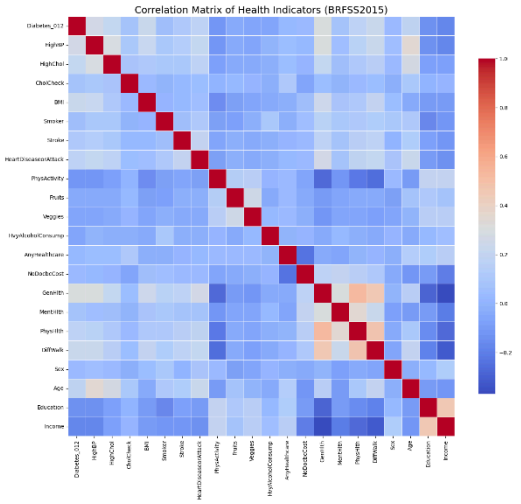

Fig. 4. Features Correlation Matrix

Here, we can focus on the most top row which is our target. Since none of the color shows distinct effect to the target feature, none of the features will be removed in the next step.

C. Data Preprocessing

After learning about our data, we will next prepare them for the modelling. First separate the target variable into y variable and keep the other attributes in x. Second, since we expect 3 outputs, encode the data into 3 categories. Third, split data into training and testing data, here we will use 80:20 ratio using a stratified split approach to preserve the original class distribution in both subsets.

From our previous step we informed that our data is not normally distributed therefore we will do 2 more steps in the data processing to minimize the effect of data imbalance. The target variable is encoded using one hot encoding to support multi class classification. In addition, class weights are computed using an inverse frequency method to mitigate bias toward the majority class and to enhance the model's ability to detect minority class samples.

D. Modelling Using Multi-Layer Perceptron

Multi-Layer Perceptron or MLP is one of the well-known techniques of Deep Learning which as its name infers using multiple hidden layers[5].

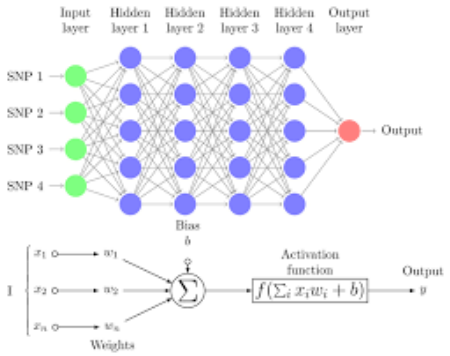

Fig. 5. Multi-Layer Perceptron (MLP) diagram with four hidden layers and a collection of single nucleotide polymorphisms (SNPs) as input and illustrates a basic "neuron" with n inputs. One neuron is the result of applying the nonlinear transformations of linear combinations (xi, wi, and biases b). These figures were redrawn from tikz code in http://www.texample.net/tikz/examples/neural-network.

The input layer receives 21 standardized features derived from lifestyle and health survey data. The network consists of three ReLU hidden layers with 128, 64, and 32 neurons respectively. This gradually decreasing structure is intended to encourage the model to learn increasingly abstract representations of the input data.

Each hidden layer uses the Rectified Linear Unit activation function due to its computational efficiency and ability to mitigate the vanishing gradient problem. To reduce the risk of overfitting, dropout regularization with a rate of 30 percent is applied after each hidden layer. The output layer consists of three neurons corresponding to the target classes and uses the Softmax activation function to produce a probability distribution over the classes.

The model is then trained with 50 epochs and batch size of 512.

E. Modelling Using XGBoost

XGBoost (Extreme Gradient Boosting) is a scalable and highly efficient implementation of gradient-boosted decision trees. It works by sequentially building an ensemble of weak learners—specifically regression trees—where each new tree attempts to correct the errors (residuals) made by the previous ones [6]. It is widely recognized in research for its speed and performance due to its use of advanced regularization techniques, which prevent model overfitting, and its ability to handle missing data and non-linear relationships effectively.

In this research, to handle class imbalance within the dataset, we implemented a custom sample weighting strategy. Each training instance was assigned a weight derived from class_weights, ensuring that the model placed higher importance on underrepresented classes during the optimization process. The model was trained using the hist tree method, which leverages histogram-based binning to accelerate the training process on large datasets.

The model was configured with specific hyperparameters to balance complexity and generalization. We utilized 2,000 estimators with a conservative learning rate of 0.02 to ensure a detailed gradient descent. To control tree depth and prevent overfitting, max_depth was set to 5, while gamma and reg_alpha were introduced to provide further regularization. Furthermore, we employed stochastic elements by setting both subsample and colsample_bytree to

0.8, meaning only 80% of the data and features were used for each boosting round to increase the model's robustness.

III. Results and Conclusion

A. Multi Layer Perceptron

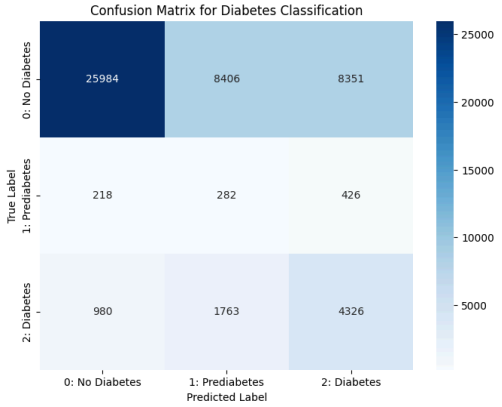

Fig. 6. Multi-Layer Perceptron (MLP) Model Confusion Matrix

Analysis of the confusion matrix indicates that the Multi-Layer Perceptron (MLP) model exhibits significant limitations in accurately identifying diabetes and prediabetes conditions. Although the model correctly identified 25,984 healthy individuals, the recall competence (sensitivity) for minority classes is notably poor. Specifically, the "Prediabetes" class achieved a recall rate of only 30.4%, with a mere 282 cases correctly detected out of 926 total instances. Similarly, for the "Diabetes" class, the model failed to detect nearly 39% of positive cases, resulting in a recall of 61.2% (4,326 correct predictions out of 7,069). These results suggest that the MLP architecture struggles to differentiate subtle feature variations between classes, often misclassifying ill individuals as healthy or in an intermediate stage.

From a clinical perspective, these findings demonstrate that the current MLP model is insufficient for medical conclusions or health screening purposes. The high rate of false negatives—particularly the failure to identify 69.6% of prediabetic patients and 38.8% of diabetic patients—poses a grave risk to patient safety, as these individuals would be denied critical early intervention. In medical informatics, a model must prioritize high sensitivity to ensure no cases are missed; however, this model's tendency to overlook a substantial portion of the at-risk population renders it unreliable for real-world healthcare applications where diagnostic accuracy is paramount.

B. XGBoost

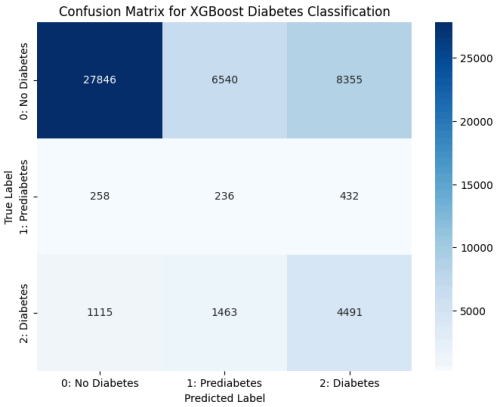

Fig. 7. XGBoost Model Confusion Matrix

Analysis of the confusion matrix for the XGBoost model reveals that while it demonstrates superior overall accuracy—particularly in identifying healthy individuals—it continues to exhibit significant shortcomings in capturing minority classes. The model correctly identifies 27,846 "No Diabetes" cases, a notable improvement over the MLP architecture. However, its recall competence for the most critical categories remains alarmingly low. For the "Prediabetes" (Class 1) category, the model achieved a recall of only 25.5%, correctly identifying just 236 out of 926 actual cases—a performance that is paradoxically lower than the MLP's sensitivity for this class. For "Diabetes" (Class 2), the recall improved slightly to 63.5% (4,491 correct predictions out of 7,069), but the model still misclassified over 2,500 diabetic patients as either healthy or prediabetic.

From a clinical standpoint, these results reinforce that neither architecture is currently suitable for making definitive medical conclusions or serving as a standalone diagnostic tool. The XGBoost model's high rate of False Negatives—missing 74.5% of prediabetic patients and 36.5% of diabetic patients—presents a severe risk in a healthcare setting. Failing to diagnose these individuals means missing the "window of opportunity" for early intervention and lifestyle modifications that could prevent disease progression or life-threatening complications. While XGBoost is more robust in recognizing healthy patterns, its inability to reliably isolate diseased states signifies that the model cannot yet meet the high-stakes demands of medical screening where sensitivity is the most vital metric for patient safety.

C. Application

As the media to connect users and the model, a website is chosen as an effective approach compared to mobile/desktop application. Here we UI design an application called "DiaCare".

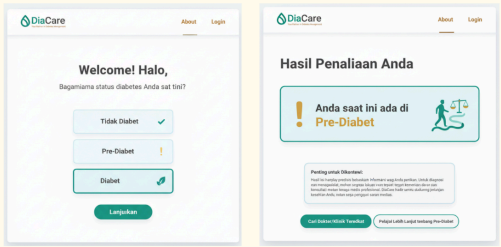

Fig. 8. Website Interface Design

Though the website is not yet made, user can enter their identity in ipynb using manual input.

```
============================================================
       DIAGNOSA DIABETES AI (INPUT MANUAL)
============================================================

[A. RIWAYAT KESEHATAN (0=Tidak, 1=Ya)]
 1. Tekanan darah tinggi? : 0
 2. Kolesterol tinggi? : 0
 3. Cek kolesterol 5thn terakhir? : 0
 4. Perokok (min 100 btg)? : 0
 5. Pernah stroke? : 0
 6. Sakit jantung/serangan? : 0
 7. Olahraga rutin (30 hari)? : 0
 8. Punya asuransi? : 0
 9. Batal ke dokter krn biaya? : 0
10. Susah jalan/naik tangga? : 0

[B. GAYA HIDUP]
Info BMI: 24.0 (Normal), 28.0 (Gemuk), 32.0 (Obesitas)
11. BMI (angka desimal, cth 24.5) : 29
12. Makan buah tiap hari? (0/1) : 0
13. Makan sayur tiap hari? (0/1) : 1
14. Alkohol berat? (0/1) : 0

Info Kesehatan Umum (1-5): 1=Sangat Sehat ... 5=Sangat Buruk
15. Kesehatan Umum (1-5) : 3
16. Hari mental buruk (0-30) : 15
17. Hari fisik buruk (0-30) : 3

[C. DATA DIRI]
18. Kelamin (0=Cewek, 1=Cowok) : 0
Info Umur: 1(18-24th) ... 9(60-64th) ... 13(80+)
19. Kategori Umur (1-13) : 1
Info Edukasi: 4(SMA), 5(D3/Mhs), 6(S1+)
20. Pendidikan (1-6) : 5
Info Gaji: 1-4(Bawah), 5-7(Menengah), 8(Atas)
21. Skala Gaji (1-8) : 4

============================================================
  HASIL PREDIKSI: TIDAK ADA DIABETES (SEHAT)
============================================================
```

Fig. 9. User Interactive Prediction

IV. Conclusion

This study evaluated the efficacy of Multi-Layer Perceptron (MLP) and XGBoost architectures in predicting diabetes risk using lifestyle and health indicators from the BRFSS dataset. The comparative analysis reveals that while both models achieved high accuracy in identifying healthy individuals, they faced significant challenges in detecting minority classes due to data imbalance. Specifically, the XGBoost model demonstrated superior overall stability and a slightly higher recall for the "Diabetes" class at 63.5%, compared to 61.2% for the MLP model. However, the MLP model performed marginally better in detecting "Prediabetes" cases, though both models exhibited critically low sensitivity in this category (30.4% for MLP and 25.5% for XGBoost).

From a clinical perspective, the current iteration of these models is not yet suitable for deployment as a definitive diagnostic tool. The high rate of False Negatives, particularly in the prediabetes category, indicates a risk of missing patients who require early intervention. Reliable medical informatics requires a higher degree of sensitivity to ensure patient safety and effective treatment planning.

Despite these clinical limitations, the proposed "DiaCare" application serves a vital purpose as a preliminary self-assessment and awareness tool. By utilizing non-invasive lifestyle data; such as BMI, physical activity, and age, the model provides users with an accessible means to evaluate their potential risk profile. Even with moderate predictive power, the tool functions effectively to heighten user vigilance, encouraging individuals with flagged risk factors to seek professional medical advice and adopt healthier lifestyle choices. Therefore, while not a replacement for laboratory testing, this system acts as a crucial first line of defense in public health education and proactive disease prevention.

References

[1] Simbolon D , Siregar A , Talib RA , et al. Physiological Factors and Physical Activity Contribute to the Incidence of Type 2 Diabetes Mellitus in Indonesia. Kesmas. 2020; 15(3): 120-127 DOI: 10.21109/kesmas.v15i3.3354 Available at: https://scholarhub.ui.ac.id/kesmas/vol15/iss3/6

[2] Matharaarachchi, S., Domaratzki, M., & Muthukumarana, S. (2024). Enhancing SMOTE for imbalanced data with abnormal minority instances. Machine Learning With Applications, 18, 100597. https://doi.org/10.1016/j.mlwa.2024.100597

[3] Teboul, A. (2014). Diabetes Health Indicators Dataset, The underlying uncleaned data comes from the CDC's BRFSS 2015. Kaggle. Retrieved January 4, 2026, from https://www.kaggle.com/datasets/alexteboul/diabetes-health-indicators-dataset

[4] Muraina, I. O., Adesanya, O. M., Agoi, M. A., & Abam, S. O. (2023). The Necessity of Exploratory Data Analysis How are preprocessing activities beneficial to Data Analysts and Professional Researchers in Academia. International Journal of Scientific Research in Computer Sciences and Engineering, 11(3), 22–28. https://doi.org/10.26438/ijsrcse/v11i3.2228

[5] Panchal, G., Ganatra, A., Kosta, Y. P., & Panchal, D. (2011). Behaviour analysis of multilayer perceptronswith multiple hidden neurons and hidden layers. International Journal of Computer Theory and Engineering, 332–337. https://doi.org/10.7763/ijcte.2011.v3.328

[6] Ogunleye, A., & Wang, Q. (2019). XGBoost Model for Chronic Kidney Disease Diagnosis. IEEE/ACM Transactions on Computational Biology and Bioinformatics, 17(6), 2131–2140. https://doi.org/10.1109/tcbb.2019.

Appendix

A. Code

https://colab.research.google.com/drive/107Bmlz0VmXxO9Yi-zmUWvRZJ4-O9TpKt?usp=sharing

```
# -*- coding: utf-8 -*-
"""AoL Ai

Automatically generated by Colab.

Original file is located at

https://colab.research.google.com/drive/107Bmlz0VmXxO9Yi-zmUWvRZJ4-O9TpKt
"""

# Import untuk menghubungkan Google Drive
from google.colab import drive

# Import Library Data Handling
import pandas as pd
import numpy as np

# Import Library Preprocessing dan Sklearn
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.utils.class_weight import compute_class_weight

# Import Library Deep Learning (TensorFlow/Keras)
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.optimizers import Adam

# Import Library Evaluasi
from sklearn.metrics import classification_report, confusion_matrix, f1_score

# 1. Mount Google Drive
```

```
drive.mount('/content/drive')

# 2. Definisikan path ke file Anda
file_path = '/content/drive/MyDrive/AOLAI/diabetes_012_health_indicators_BRFSS2015.csv'

# 3. Baca data ke dalam DataFrame pandas
df = pd.read_csv(file_path)

# Tampilkan 5 baris pertama untuk memastikan data berhasil dimuat
print(df.head())

"""## Data Understanding"""

# --- 1. Jumlah Baris dan Kolom ---
print("--- 1. Dimensi Dataset ---")
jumlah_baris = df.shape[0]
jumlah_kolom = df.shape[1]
print(f"Jumlah Baris (Survei Respons): {jumlah_baris:,}")
print(f"Jumlah Kolom (Fitur + Target): {jumlah_kolom}")
print("-" * 30)

# --- 2. Informasi Kolom dan Tipe Data ---
print("--- 2. Info Kolom dan Tipe Data ---")
print(df.info())
print("-" * 30)

# --- 3. Persebaran Nilai (Unik dan Deskriptif) ---

# Tampilkan nilai unik untuk kolom bertipe kategori/ordinal
categorical_cols = ['Diabetes_012', 'HighBP', 'HighChol', 'CholCheck', 'Smoker', 'Stroke',
                    'HeartDiseaseorAttack', 'PhysActivity', 'Fruits', 'Veggies',
                    'HvyAlcoholConsump', 'AnyHealthcare', 'NoDocbcCost', 'DiffWalk', 'Sex']
print("--- 3.1. Persebaran Nilai Kategori/Ordinal ---")
for col in categorical_cols:
    print(f"\n{col} (Nilai Unik):")
    # Hitung nilai unik dan persentase, urutkan berdasarkan index
    value_counts = df[col].value_counts().sort_index()
    percentage = (df[col].value_counts(normalize=True) * 100).sort_index().round(2)

    # Gabungkan menjadi satu DataFrame
    summary = pd.DataFrame({'Count': value_counts, 'Percentage': percentage.astype(str) + ' %'})

    # Sesuaikan label untuk Diabetes_012 agar mudah dipahami
    if col == 'Diabetes_012':
        summary.index = ['0: No Diabetes/Pregnancy', '1: Prediabetes', '2: Diabetes']

    print(summary)

print("\n--- 3.2. Statistik Deskriptif (Fitur Numerik) ---")
# Menampilkan statistik deskriptif untuk fitur numerik
print(df[['BMI', 'GenHlth', 'MentHlth', 'PhysHlth', 'Age', 'Education', 'Income']].describe().round(2))
print("-" * 30)

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# --- ASUMSI: df (DataFrame) sudah dimuat di sini ---
# Gunakan DataFrame 'df' yang asli (sebelum di-clip outlier)
# Jika Anda menjalankan ini di Google Colab, pastikan df sudah dimuat.
# Misalnya: df = pd.read_csv(file_path)

print("--- Pembuatan Histogram Fitur Non-Biner Dimulai ---")

# Identifikasi kolom yang bukan hanya biner (lebih dari 2 nilai unik)
# Kita juga mengecualikan kolom target
binary_cols = ['HighBP', 'HighChol', 'CholCheck', 'Smoker', 'Stroke',
               'HeartDiseaseorAttack', 'PhysActivity', 'Fruits', 'Veggies',
               'HvyAlcoholConsump', 'AnyHealthcare', 'NoDocbcCost', 'DiffWalk', 'Sex']

all_cols = df.columns.tolist()
target_col = 'Diabetes_012'

# Fitur yang akan diplot (Non-Biner/Kontinu/Ordinal)
# Ini termasuk BMI, GenHlth, MentHlth, PhysHlth, Age, Education, Income, dan target
feature_cols = [col for col in all_cols if col not in binary_cols and col != target_col]
feature_cols.append(target_col) # Tambahkan target untuk melihat distribusinya

# --- 1. Hitung dan Tampilkan Skewness ---
print("\n--- Nilai Skewness Fitur Non-Biner ---")
# Hitung skewness hanya untuk fitur yang akan diplot
skewness_results = df[feature_cols].skew().sort_values(ascending=False)
print(skewness_results)
print("-" * 40)

# --- Membuat Histogram untuk Setiap Fitur Non-Biner ---
num_features = len(feature_cols)
# Atur tata letak subplot (misalnya 4 kolom)
n_cols = 4
n_rows = int(np.ceil(num_features / n_cols))

plt.figure(figsize=(18, 5 * n_rows))
plt.suptitle('Distribusi (Histogram) Fitur Non-Biner/Ordinal', fontsize=18, y=1.02)

for i, col in enumerate(feature_cols):
    plt.subplot(n_rows, n_cols, i + 1)

    # Dapatkan nilai skewness untuk digunakan di judul plot
    skew_val = skewness_results[col]

    # Atur bins secara otomatis
    # Untuk fitur seperti MentHlth (0-30), bins bisa disesuaikan.
    if col in ['MentHlth', 'PhysHlth']:
        sns.histplot(df[col], bins=31, kde=False, color='skyblue', edgecolor='black')
        plt.xticks(range(0, 31, 5)) # Tampilkan label setiap 5 hari
    elif col == 'BMI':
        sns.histplot(df[col], bins=50, kde=True, color='lightcoral', edgecolor='black')
    else:
        # Untuk fitur ordinal (Age, Education, Income, GenHlth) dan Target
```

```
                            sns.histplot(df[col],
bins=len(df[col].unique()),              kde=False,
color='lightgreen', edgecolor='black')
        plt.xticks(df[col].unique())

    # Tampilkan nilai skewness di judul plot
        plt.title(f'Distribusi:   {col}   (Skewness:
{skew_val:.2f})', fontsize=14)
    plt.xlabel(col)
    plt.ylabel('Frekuensi')

plt.tight_layout(rect=[0,   0.03,   1,   0.98])   #
Sesuaikan layout agar tidak tumpang tindih
plt.show()

# --- 4. Cek Missing Value ---
print("--- 4. Cek Missing Values ---")
print(df.isnull().sum())
print("-" * 30)

import matplotlib.pyplot as plt
import seaborn as sns

# Pastikan kita menggunakan DataFrame 'df' sebelum
di-scale
# Kita akan menggunakan korelasi Pearson
correlation_matrix = df.corr(method='pearson')

# --- 1. Korelasi Fitur vs. Target (Diabetes_012)
---
print("--- 1. Korelasi Pearson dengan Diabetes_012
---")

# Ambil baris korelasi target, urutkan dari yang
paling kuat
target_corr                                          =
correlation_matrix['Diabetes_012'].sort_values(asce
nding=False)

# Hilangkan korelasi dengan dirinya sendiri (yang
bernilai 1)
target_corr = target_corr.drop('Diabetes_012')

print(target_corr)
print("-" * 50)

#   ---   2.   Visualisasi   Full   Correlation   Matrix
(Heatmap) ---
plt.figure(figsize=(16, 14))
#   Gunakan   'viridis'   atau   'coolwarm'   untuk
visualisasi yang baik
sns.heatmap(
    correlation_matrix,
    annot=False,
    fmt=".2f",
    cmap='coolwarm',
    linewidths=.5,
    cbar_kws={'shrink': .8}
)
plt.title('Correlation Matrix of Health Indicators
(BRFSS2015)', fontsize=18)
plt.show()
#

"""## Data Preprocessing"""

# --- 1. Pemisahan Fitur (X) dan Target (y) ---
X = df.drop('Diabetes_012', axis=1) # Semua kolom
kecuali target
y = df['Diabetes_012']

# --- 2. Target Encoding (One-Hot Encoding) ---
y_encoded = to_categorical(y)
num_classes = y_encoded.shape[1]
print(f"Target   ter-encode   menjadi   {num_classes}
kolom.")

# --- 3. Split Data: Training dan Testing ---
# Kita gunakan 80% untuk training dan 20% untuk
testing
# Stratify=y memastikan proporsi kelas (imbalance)
dipertahankan di set training dan testing
X_train,    X_test,    y_train_cat,    y_test_cat    =
train_test_split(
    X,
    y_encoded,
    test_size=0.2,
    random_state=42,
    stratify=y
)
print(f"Ukuran Data Training (X): {X_train.shape}")
print(f"Ukuran Data Testing (X): {X_test.shape}")

#  Kita  juga  simpan  y_train/y_test  yang  belum
di-encode untuk menghitung Class Weights
_, _, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# --- 4. Feature Scaling (Standarisasi) ---

# Tentukan kolom yang akan di-scale
# Kolom yang sudah biner (0 atau 1) tidak perlu
di-scale (HighBP, HighChol, etc.)
# Kita fokus pada BMI, GenHlth, MentHlth, PhysHlth,
Age, Education, Income
scale_cols   =   ['BMI',   'GenHlth',   'MentHlth',
'PhysHlth', 'Age', 'Education', 'Income']

# Inisialisasi Scaler
scaler = StandardScaler()

# Lakukan fit pada data training, lalu transform
data training dan testing
X_train[scale_cols]                                  =
scaler.fit_transform(X_train[scale_cols])
X_test[scale_cols]                                   =
scaler.transform(X_test[scale_cols])

print("\nFitur   telah   berhasil   di-Standarisasi
(Scaled).")
print(X_train[scale_cols].head())   #   Cek   hasil
scaling

#   ---   5.   Menghitung   Class   Weights   (Mengatasi
Imbalance) ---

#   Hitung   bobot   kelas   menggunakan   frekuensi
kebalikan (inverse frequency)
#   Ini   adalah   kunci   untuk   memastikan   model
memperhatikan kelas Prediabetes (1) dan Diabetes
(2)
weights = compute_class_weight(
    class_weight='balanced',
    classes=np.unique(y_train),
    y=y_train
)

#  Konversi  array  weights  ke  dictionary,  yang
dibutuhkan oleh Keras
class_weights = dict(enumerate(weights))

print("\n--- Class Weights (Untuk Imbalance) ---")
print(f"Kelas        0        (No        Diabetes):
{class_weights.get(0):.4f}")
print(f"Kelas            1           (Prediabetes):
{class_weights.get(1):.4f}")
print(f"Kelas             2             (Diabetes):
{class_weights.get(2):.4f}")
```

```
"""##Modeling"""

input_dim = X_train.shape[1]

# --- 1. Membangun Model MLP ---
model = Sequential([
    # Lapisan Input & Lapisan Tersembunyi Pertama
    # Neuron 128, Aktivasi ReLU
                Dense(128,       activation='relu',
input_shape=(input_dim,)),
    # Dropout untuk mencegah overfitting (mematikan
30% neuron secara acak)
    Dropout(0.3),

    # Lapisan Tersembunyi Kedua
    # Neuron 64, Aktivasi ReLU
    Dense(64, activation='relu'),
    Dropout(0.3),

    # Lapisan Tersembunyi Ketiga
    # Neuron 32, Aktivasi ReLU
    Dense(32, activation='relu'),
    Dropout(0.3),

    # Lapisan Output
     # Neuron 3 (sesuai jumlah kelas), Aktivasi
Softmax untuk probabilitas multi-kelas
    Dense(num_classes, activation='softmax')
])

# --- 2. Kompilasi Model ---
# Optimizer: Adam (standar yang bagus)
# Loss: Categorical Crossentropy (untuk klasifikasi
multi-kelas dengan One-Hot Encoding)
# Metrics: Focus pada Recall dan F1-Score karena
ada imbalance
model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='categorical_crossentropy',
                               metrics=['accuracy',
tf.keras.metrics.Recall(name='recall'),
tf.keras.metrics.F1Score(name='f1_score')]
)

# Tampilkan ringkasan arsitektur model
print("--- Arsitektur Model MLP ---")
model.summary()
print("-" * 35)

# --- 3. Pelatihan Model (Training) ---

# Tentukan parameter training
EPOCHS = 50
BATCH_SIZE = 512

print("\n--- Model Training Dimulai ---")

history = model.fit(
    X_train,
    y_train_cat,
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    validation_split=0.1,
    class_weight=class_weights,
    verbose=1
)

"""## Evaluation"""

# --- 1. Plot Metrik Pelatihan ---
print("--- 1. Grafik Sejarah Pelatihan ---")

# Fungsi untuk membuat plot
def plot_training_history(history):
    fig, axes = plt.subplots(1, 2, figsize=(15, 5))

    # Plot Loss
              axes[0].plot(history.history['loss'],
label='Training Loss')
          axes[0].plot(history.history['val_loss'],
label='Validation Loss')
    axes[0].set_title('Loss over Epochs')
    axes[0].set_xlabel('Epoch')
    axes[0].set_ylabel('Loss')
    axes[0].legend()

    # Plot Accuracy
          axes[1].plot(history.history['accuracy'],
label='Training Accuracy')
       axes[1].plot(history.history['val_accuracy'],
label='Validation Accuracy')
    axes[1].set_title('Accuracy over Epochs')
    axes[1].set_xlabel('Epoch')
    axes[1].set_ylabel('Accuracy')
    axes[1].legend()

    plt.show()

plot_training_history(history)

# --- 2. Evaluasi pada Data Testing ---

# Melakukan prediksi pada data testing
y_pred_cat = model.predict(X_test)
# Mengubah hasil prediksi dari one-hot encoding
kembali ke label kelas (0, 1, 2)
y_pred = np.argmax(y_pred_cat, axis=1)

# Mengubah y_test_cat (one-hot) kembali ke label
kelas (0, 1, 2)
y_true = np.argmax(y_test_cat, axis=1)

# Hitung metrik F1-Score (penting untuk imbalance)
weighted_f1      =      f1_score(y_true,      y_pred,
average='weighted')

print("\n---      2.1.      Laporan      Klasifikasi
(Classification Report) ---")
print(classification_report(y_true,          y_pred,
target_names=['0: No Diabetes', '1: Prediabetes',
'2: Diabetes']))
print(f"F1-Score (Weighted): {weighted_f1:.4f}")
print("-" * 50)

# --- 3. Confusion Matrix ---
print("\n--- 3. Confusion Matrix ---")

cm = confusion_matrix(y_true, y_pred)
labels = ['0: No Diabetes', '1: Prediabetes', '2:
Diabetes']

plt.figure(figsize=(8, 6))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=labels,
    yticklabels=labels
)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion     Matrix     for     Diabetes
Classification')
plt.show()
#

!pip install catboost

# --- 1. Inisialisasi Model XGBoost ---

from xgboost import XGBClassifier
from catboost import CatBoostClassifier, Pool
```

```
model_xgb = XGBClassifier(
    #objective='multi:softmax',
        n_estimators=2000,
    learning_rate=0.02,
    max_depth=5,
    min_child_weight=3,
    subsample=0.8,
    colsample_bytree=0.8,
    gamma=1,
    reg_alpha=0.1,
    reg_lambda=1,
    objective='multi:softmax',
    eval_metric='auc',
    tree_method='hist',
    verbosity=0,
    enable_categorical=True
)

print("--- Model XGBoost Dibuat ---")

# --- 2. Pelatihan Model (Training) ---
print("\n--- Model XGBoost Training Dimulai ---")

# mapping class label to its weight
weights_map     =     {0:     class_weights[0],     1:
class_weights[1], 2: class_weights[2]}
#  create  a  list  of  weights  for  each  sample  in
y_train
sample_weights_array  = np.array([weights_map[label]
for label in y_train])

model_xgb.fit(X_train,                        y_train,
sample_weight=sample_weights_array)

print("--- Model XGBoost Training Selesai ---")

# --- 3. Evaluasi pada Data Testing ---
print("\n--- 3.1. Prediksi pada Data Testing ---")
y_pred_xgb = model_xgb.predict(X_test)

print("\n---      3.2.      Laporan      Klasifikasi
(Classification Report) XGBoost ---")
print(classification_report(y_test,      y_pred_xgb,
target_names=['0:  No  Diabetes',  '1:  Prediabetes',
'2: Diabetes']))

print("\n--- 3.3. Confusion Matrix XGBoost ---")
cm_xgb = confusion_matrix(y_test, y_pred_xgb)

plt.figure(figsize=(8, 6))
sns.heatmap(
    cm_xgb,
    annot=True,
    fmt='d',
    cmap='Blues',
          xticklabels=['0:    No    Diabetes',    '1:
Prediabetes', '2: Diabetes'],
          yticklabels=['0:    No    Diabetes',    '1:
Prediabetes', '2: Diabetes']
)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion  Matrix  for  XGBoost  Diabetes
Classification')
plt.show()

print("\n--- 3.1. Prediksi pada Data Testing ---")
y_pred_xgb = model_xgb.predict(X_test)

from sklearn.metrics import f1_score
weighted_f1_xgb   =   f1_score(y_test,   y_pred_xgb,
average='weighted')

print(classification_report(y_test,      y_pred_xgb,
target_names=['0:  No  Diabetes',  '1:  Prediabetes',
'2: Diabetes']))
print(f"F1-Score         (Weighted         Average):
{weighted_f1_xgb:.4f}")

import pandas as pd
import numpy as np

#  ---  1.  DEFINISI  FUNGSI  PREDIKSI  (YANG  TADI
HILANG/ERROR) ---
def prediksi_diabetes(input_data):

    new_df = pd.DataFrame([input_data])

    try:
        feature_order = X.columns
        new_df = new_df[feature_order]
    except NameError:
            print("Error:  Data  training  (X)  tidak
ditemukan.    Pastikan    proses    training    sudah
dijalankan.")
        return

     scale_cols  =  ['BMI',  'GenHlth',  'MentHlth',
'PhysHlth', 'Age', 'Education', 'Income']
                       new_df[scale_cols]          =
scaler.transform(new_df[scale_cols])

     # Lakukan Prediksi (Menggunakan model XGBoost
yang sudah dilatih)
    try:
        prediction = model_xgb.predict(new_df)[0]
    except NameError:
         # Fallback jika model_xgb belum ada, coba
pakai model Neural Network
        try:
            pred_prob = model.predict(new_df)
                 prediction  =  np.argmax(pred_prob,
axis=1)[0]
        except:
               print("Error: Model tidak ditemukan.
Harap jalankan training model dulu.")
            return

    # Mapping hasil angka ke label teks
    label_map = {
        0: 'TIDAK ADA DIABETES (SEHAT) ✅',
        1: 'PREDIABETES (WASPADA) ⚠️',
        2: 'DIABETES ❗'
    }

    result = label_map.get(prediction, "Unknown")

    print("\n" + "="*50)
    print(f"  HASIL PREDIKSI: {result}")
    print("="*50)
    return result

# --- 2. DEFINISI FUNGSI INPUT INTERAKTIF ---
def      get_user_input(prompt,      type_func=int,
min_val=None, max_val=None):
    while True:
        try:
            raw = input(prompt)
             if not raw.strip(): return 0 # Default
0 jika kosong (opsional)
            val = type_func(raw)
                if  min_val  is  not  None  and  val  <
min_val:
                        print(f"     ❌  Nilai  min:
{min_val}")
                continue
                if  max_val  is  not  None  and  val  >
max_val:
```

```
                    print(f"    ❌ Nilai max: {max_val}")
                continue
            return val
        except ValueError:
                print("    ❌ Masukkan angka yang valid.")

def mulai_diagnosa_interaktif():
    print("\n" + "="*60)
    print("       DIAGNOSA DIABETES AI")
    print("="*60)

    data = {}

       print("\n[A. RIWAYAT KESEHATAN (0=Tidak, 1=Ya)]")
     data['HighBP'] = get_user_input(" 1. Tekanan darah tinggi? : ", int, 0, 1)
        data['HighChol'] = get_user_input(" 2. Kolesterol tinggi? : ", int, 0, 1)
      data['CholCheck'] = get_user_input(" 3. Cek kolesterol 5thn terakhir? : ", int, 0, 1)
     data['Smoker'] = get_user_input(" 4. Perokok (min 100 btg)? : ", int, 0, 1)
      data['Stroke'] = get_user_input(" 5. Pernah stroke? : ", int, 0, 1)
    data['HeartDiseaseorAttack'] = get_user_input(" 6. Sakit jantung/serangan? : ", int, 0, 1)
      data['PhysActivity'] = get_user_input(" 7. Olahraga rutin (30 hari)? : ", int, 0, 1)
      data['AnyHealthcare'] = get_user_input(" 8. Punya asuransi? : ", int, 0, 1)
    data['NoDocbcCost'] = get_user_input(" 9. Batal ke dokter krn biaya? : ", int, 0, 1)
     data['DiffWalk'] = get_user_input("10. Susah jalan/naik tangga? : ", int, 0, 1)

    print("\n[B. GAYA HIDUP]")
     print("Info BMI: 24.0 (Normal), 28.0 (Gemuk), 32.0 (Obesitas)")
     data['BMI'] = get_user_input("11. BMI (angka desimal, cth 24.5) : ", float, 10, 100)
    data['Fruits'] = get_user_input("12. Makan buah tiap hari? (0/1) : ", int, 0, 1)
      data['Veggies'] = get_user_input("13. Makan sayur tiap hari? (0/1) : ", int, 0, 1)
    data['HvyAlcoholConsump'] = get_user_input("14. Alkohol berat? (0/1) : ", int, 0, 1)

     print("\nInfo Kesehatan Umum (1-5): 1=Sangat Sehat ... 5=Sangat Buruk")
    data['GenHlth'] = get_user_input("15. Kesehatan Umum (1-5) : ", int, 1, 5)
      data['MentHlth'] = get_user_input("16. Hari mental buruk (0-30) : ", int, 0, 30)
      data['PhysHlth'] = get_user_input("17. Hari fisik buruk (0-30) : ", int, 0, 30)

    print("\n[C. DATA DIRI]")
       data['Sex'] = get_user_input("18. Kelamin (0=Cewek, 1=Cowok) : ", int, 0, 1)

    print("Info Umur: 1(18-24th) ... 9(60-64th) ... 13(80+)")
    data['Age'] = get_user_input("19. Kategori Umur (1-13) : ", int, 1, 13)

       print("Info Edukasi: 4(SMA), 5(D3/Mhs), 6(S1+)")
        data['Education'] = get_user_input("20. Pendidikan (1-6) : ", int, 1, 6)

     print("Info Gaji: 1-4(Bawah), 5-7(Menengah), 8(Atas)")
    data['Income'] = get_user_input("21. Skala Gaji (1-8) : ", int, 1, 8)

    # PANGGIL FUNGSI PREDIKSI DI SINI
    prediksi_diabetes(data)

# --- 3. JALANKAN PROGRAM ---
mulai_diagnosa_interaktif()
```